

СИГНАЛЫ И СЛОТЫ Qt/QML - ПОЛНОЕ РУКОВОДСТВО

Что это такое?

Сигналы и слоты - это механизм связи между объектами в Qt.

Аналогия:

- **Сигнал** = кнопка на пульте ДУ 
- **Слот** = функция которая выполняется когда нажата кнопка

Ключевое: Сигнал НЕ ЗНАЕТ кто его слушает! Это развязка (decoupling).

ЧАСТЬ 1: СИГНАЛЫ В C++

Объявление сигнала в .h файле:

```
// timeline.h
class Timeline : public QObject {
    Q_OBJECT // ОБЯЗАТЕЛЬНО для сигналов/слотов!

signals: // ВСЕГДА public!
    // Сигналы НЕ имеют тела - только объявление
    void clipAdded(int index);
    void clipRemoved(int index);
    void currentTimeChanged();
    void renderProgress(int percent);

    // Можно с параметрами любых типов
    void errorOccurred(QString message, int code);
};
```

Вызов (emit) сигнала в .cpp:

```
// timeline.cpp
bool Timeline::addClip(const QString& filepath, int trackIndex, double
startTime) {
    // ... добавляем клип ...

    m_clips.append(clip);

    // ВЫЗЫВАЕМ сигнал - все кто подключен получают уведомление!
    emit clipAdded(m_clips.size() - 1); // emit - ключевое слово Qt
    emit clipsChanged();

    return true;
}
```

⚠ **ВАЖНО:** emit - это макрос Qt, он ничего не делает, просто читабельность!

🔗 ЧАСТЬ 2: СЛОТЫ В C++

Объявление слота в .h:

```
class Timeline : public QObject {
    Q_OBJECT

public slots: // Могут быть public, protected, private
    // Слоты = обычные функции, которые можно подключить к сигналам
    void setCurrentTime(double time);
    bool addClip(const QString& filepath, int trackIndex, double startTime);

    // Слот может быть без параметров
    void play();
    void pause();
    void stop();
};
```

Реализация в .cpp:

```
void Timeline::setCurrentTime(double time) {
    if (m_currentTime != time) {
        m_currentTime = time;
        emit currentTimeChanged(); // Сигнал об изменении
    }
}

void Timeline::play() {
    m_isPlaying = true;
    emit playbackStateChanged();
}
```

🔗 ЧАСТЬ 3: ПОДКЛЮЧЕНИЕ В C++

Способ 1: connect() в старом стиле (НЕ рекомендуется):

```
// НЕ ДЕЛАЙ ТАК! Устаревший способ
connect(timeline, SIGNAL(clipAdded(int)),
        this, SLOT(onClipAdded(int)));
```

Способ 2: connect() новый стиль (✓ ИСПОЛЬЗУЙ):

```
// main.cpp или конструктор
Timeline timeline;
FFmpegHelper ffmpeg;

// Подключаем сигнал timeline к слоту ffmpeg
connect(&timeline, &Timeline::clipAdded,    // откуда сигнал
        &ffmpeg, &FFmpegHelper::loadClip); // куда слот

// С лямбдой (анонимная функция)
connect(&timeline, &Timeline::renderProgress,
        [](int percent) {
            qDebug() << "Прогресс:" << percent << "%";
        });

// К обычной функции
connect(&timeline, &Timeline::errorOccurred,
        &MyClass::handleError);
```

Способ 3: connect() с контекстом (безопасный):

```
// Если QML объект удалится, соединение автоматически разорвётся
connect(&timeline, &Timeline::clipAdded,
        qmlObject, [qmlObject](int index) {
            // Безопасно использовать qmlObject
        });
```

ЧАСТЬ 4: СИГНАЛЫ В QML

Объявление своего сигнала:

```
// MyComponent.qml
Rectangle {
    // Объявляем сигнал
    signal clicked()
    signal valueChanged(real newValue)
    signal clipDropped(string filepath, real time)

    MouseArea {
        anchors.fill: parent
        onClicked: {
            // ВЫЗЫВАЕМ сигнал
            parent.clicked() // НЕ нужен emit!
        }
    }
}
```

Использование сигнала:

```
// main.qml
MyComponent {
    // Способ 1: on<SignalName> обработчик
    onClicked: {
        console.log("Компонент кликнут!")
    }

    // Способ 2: с параметрами
    onValueChanged: (newValue) => {
        console.log("Новое значение:", newValue)
    }

    // Способ 3: вызов функции
    onClipDropped: (filepath, time) => {
        clipManager.addClip(filepath, 1, time)
    }
}
```

ЧАСТЬ 5: C++ ↔ QML СВЯЗЬ

Экспорт C++ объекта в QML:

```
// main.cpp
#include <QQmlContext>

int main(int argc, char *argv[]) {
    QGuiApplication app(argc, argv);
    QQmlApplicationEngine engine;

    // Создаём C++ объект
    Timeline timeline;

    // ЭКСПОРТИРУЕМ в QML под именем "cppTimeline"
    engine.rootContext()->setContextProperty("cppTimeline", &timeline);

    engine.load(url);
    return app.exec();
}
```

Использование в QML:

```
// main.qml
Window {
    Component.onCompleted: {
        // cppTimeline доступен ВЕЗДЕ в QML!
        console.log("Клипов:", cppTimeline.clipCount)
    }

    Button {
        onClicked: {
            // Вызываем C++ слот
            cppTimeline.addClip("video.mp4", 0, 5.0)
        }
    }
}
```

🔗 ЧАСТЬ 6: ПОДКЛЮЧЕНИЕ C++ СИГНАЛОВ В QML

Способ 1: Connections объект (✓ ЛУЧШИЙ):

```
// main.qml
Window {
    // Подключаемся к сигналам C++ объекта
    Connections {
        target: cppTimeline // К какому объекту подключаемся

        // function on<SignalName>(параметры)
        function onClipAdded(index) {
            console.log("Клип добавлен в C++, индекс:", index)
            timeline.refresh()
        }

        function onClipsChanged() {
            console.log("Список клипов изменён")
        }

        function onRenderProgress(percent) {
            progressBar.value = percent
        }

        function onErrorOccurred(message, code) {
            errorDialog.show(message)
        }
    }
}
```

Способ 2: Прямой обработчик (только для root element):

```
Timeline {
    // Если Timeline - C++ класс, экспортированный как компонент
    onClipAdded: (index) => {
        console.log("Клип добавлен:", index)
    }
}
```

🎯 ЧАСТЬ 7: Q_PROPERTY

Для автоматической синхронизации используй Q_PROPERTY:

```
// timeline.h
class Timeline : public QObject {
    Q_OBJECT

    // PROPERTY = автоматическая связь с QML!
    Q_PROPERTY(double currentTime READ currentTime WRITE setCurrentTime NOTIFY
currentTimeChanged)
    Q_PROPERTY(int clipCount READ clipCount NOTIFY clipsChanged)

public:
    // Геттер
    double currentTime() const { return m_currentTime; }
    int clipCount() const { return m_clips.size(); }

public slots:
    // Сеттер
    void setCurrentTime(double time) {
        if (m_currentTime != time) {
            m_currentTime = time;
            emit currentTimeChanged(); // ОБЯЗАТЕЛЬНО emit сигнал!
        }
    }

signals:
    void currentTimeChanged();
    void clipsChanged();

private:
    double m_currentTime;
    QList<TimelineClip> m_clips;
};
```

В QML это выглядит как обычное свойство:

```

Text {
    // Автоматически обновляется когда C++ emit currentTimeChanged()!
    text: "Время: " + cppTimeline.currentTime
}

Slider {
    value: cppTimeline.currentTime

    // При изменении вызывается C++ сеттер
    onValueChanged: {
        cppTimeline.currentTime = value
    }
}

```

🔥 ЧАСТЫЕ ОШИБКИ

✗ ОШИБКА 1: Забыли Q_OBJECT

```

class Timeline : public QObject {
    // Q_OBJECT ← ЗАБЫЛИ!
signals:
    void clipAdded(int index); // НЕ БУДЕТ РАБОТАТЬ!
};

```

✓ ПРАВИЛЬНО:

```

class Timeline : public QObject {
    Q_OBJECT // ← ОБЯЗАТЕЛЬНО!
signals:
    void clipAdded(int index);
};

```

✗ ОШИБКА 2: Сигнал с телом функции

```

signals:
    void clipAdded(int index) { // ✗ ОШИБКА!
        qDebug() << "Клип добавлен";
    }

```

✓ ПРАВИЛЬНО:

```

signals:
    void clipAdded(int index); // Только объявление!

```

✗ ОШИБКА 3: Забыли emit

```

void Timeline::addClip(...) {
    m_clips.append(clip);
    clipAdded(index); // ✗ Забыли emit!
}

```

✓ ПРАВИЛЬНО:

```
void Timeline::addClip(...) {  
    m_clips.append(clip);  
    emit clipAdded(index); // ✓ С emit!  
}
```

✗ ОШИБКА 4: Неправильное имя в QML

```
Connections {  
    target: cppTimeline  
    onClipsAdded: { ... } // ✗ Неправильно! Должно быть onClipAdded  
}
```

✓ ПРАВИЛЬНО:

```
Connections {  
    target: cppTimeline  
    function onClipAdded(index) { ... } // ✓ on + имя сигнала с большой буквы  
}
```

ШПАРГАЛКА

C++ → QML:

```
emit signalName(params); // В C++
```

```
Connections {  
    target: cppObject  
    function onSignalName(params) { } // В QML  
}
```

QML → C++:

```
cppObject.slotName(params) // Вызов слота
```

```
public slots:  
    void slotName(params); // В C++
```

Property (двусторонняя связь):

```
Q_PROPERTY(Type name READ getter WRITE setter NOTIFY signal)
```

```
value: cppObject.name // Чтение  
cppObject.name = newValue // Запись (вызывает setter)
```

ГОТОВО! Теперь переходим к твоим конкретным сигналам! 🚀